

STANDARD OPERATING PROCEDURE
NASA TPSX to Xtrium
Schema Mapping & Data Transformation

Document Information

SOP Number: SOP-DS-001
Version: 1.3
Effective Date: March 29, 2026
Amendment Date: June 3, 2026
Department: Data Operations
Prepared By: Xtrium Data Team
Applies To: External Data Scraping Contractors

Purpose of this Document

This Standard Operating Procedure provides step-by-step instructions for scraping the NASA TPSX Open Materials Database and transforming the extracted data into the Xtrium Materials Graph Schema JSON format. This document must be followed exactly to ensure data integrity and schema compliance.

CONFIDENTIAL — FOR AUTHORIZED CONTRACTORS ONLY

1. Purpose & Scope

This Standard Operating Procedure (SOP) governs the end-to-end workflow for scraping the NASA Thermal Protection Systems Expert (TPSX) Open Materials Database and transforming every extracted record into a valid Xtrium Materials Graph Schema JSON object. It applies to all contractor personnel involved in this data acquisition project.

1.1 Objectives

- Capture all 1,547+ material records from the NASA TPSX database.
- Map all 1,287 CSV columns to the correct Xtrium schema fields without data loss.
- Produce one well-formed JSON file per material record.
- Deliver validated outputs ready for direct ingestion by load_to_neo4j.py.

1.2 Dataset Statistics

Category	Count	Notes
Base Columns	5	Id, Material_Name, Database_Source, Category, URL
Section Columns	12	Composition, Description, Notes, Manufacturer, and others
Unique Property Groups	133	e.g., Density, Tensile_Strength, Thermal_Conductivity
Total Property Columns (incl. variants)	~1,270	Multiple instances per group
Total CSV Columns	1,287	All must be mapped — no exceptions
Total Material Records	1,547+	One JSON output per material

2. Roles & Responsibilities

Role	Responsibility
Data Scraper	Executes web scraping scripts to pull data from NASA TPSX. Exports raw CSV files for handoff.
Data Transformation Specialist	Applies this SOP to convert each CSV row into a compliant Xtrium JSON object.
QA Reviewer	Runs the validation checklist (Section 13) against finished JSON files before submission.

Role	Responsibility
Project Lead	Resolves ambiguous mappings, approves escalations, and accepts final deliverables.

3. Non-Negotiable Operating Rules

All five rules below must be observed at all times. Violation of any rule is grounds for rejecting an entire batch.

Rule 1	NO DATA LOSS Every scraped column must appear somewhere in the output JSON. If a column has no matching schema field, place its value inside the property object's raw dictionary. Nothing may be discarded.
Rule 2	NO SCHEMA MODIFICATION Do not alter the Xtrium schema structure under any circumstances. All NASA data must fit into the existing schema fields as described in this document.
Rule 3	PROPERTY GROUPING Columns sharing a base name (e.g., Density_Value, Density_Units, Density_Source) belong to a single property object in the properties array. Do not split them into separate objects.
Rule 4	MULTIPLE INSTANCES Columns with numeric suffixes (e.g., Thermal_Conductivity_1_Value, Thermal_Conductivity_2_Value) represent the same property under different test conditions. Create one separate property object per numbered instance.
Rule 5	NULL VALUES If NASA data does not supply a value for a schema field, set it to null. Do not invent, infer, or estimate data. Use empty arrays [] for list-type fields with no data.

4. Xtrium Schema — Top-Level Structure

Every material must produce a JSON object that matches the structure below exactly. Fields not available in NASA data must still be present, set to null or [].

{

```

"material_id": null,
"material_name": "...", // REQUIRED
"canonical_name": "...",
"cas_number": null,
"uns_number": null,
"icsd_id": null,
"inchikey": null,
"external_id_type": null,
"chemical_formula": null,
"material_type": "...",
"material_subtype": null,
"grade": null,
"trade_names": [],
"is_variant": false,
"base_material_canonical_name": null,
"supplier_context": { ... },
"properties": [ ... ], // REQUIRED
"standards_referenced": [],
"compliance_references": [],
"applications_mentioned": [],
"processing_notes": "...",
"hazard_flags": [],
"availability_notes": null,
"images": [],
"sources": [ ... ] // REQUIRED
}

```

5. Top-Level Field Mappings

5.1 Identity Fields

Schema Field	NASA CSV Column	Transformation Rule
material_id	None	null — assigned by load_to_neo4j.py at import time
material_name	Material_Name	REQUIRED. Copy exactly as it appears in NASA data
canonical_name	Derived from Material_Name	Lowercase, spaces to hyphens, remove specials. e.g. "Aluminum 6061-T6" → "aluminum-6061-t6"
cas_number	None	null — NASA TPSX does not provide CAS numbers
uns_number	None	null — NASA TPSX does not provide UNS designations
icsd_id	None	null
inchikey	None	null
external_id_type	None	null

Schema Field	NASA CSV Column	Transformation Rule
chemical_formula	Composition (partial)	Extract only if Composition contains a clear chemical formula (e.g., Ti-6Al-4V). Otherwise null.

v1.1 — Amendment**v1.3 — Amendment****chemical_formula: Extraction Threshold**

The SOP rule "extract only if Composition contains a clear chemical formula" applies as follows:

Composition cell content	chemical_formula	processing_notes
Formula only(e.g. "Ti-6Al-4V")	Extract exactly as written	Also include (Rule 1)
Mixed formula + prose(e.g. "Al-Mg-Si alloy — aerospace grade")	null	Append full cell as-is
Prose only(e.g. "Carbon fiber reinforced polymer")	null	Append full cell as-is

Test: if you cannot copy the cell value directly into chemical_formula without editing it, the cell belongs in processing_notes. Do not attempt to extract a formula substring from a mixed cell.

5.2 Classification Fields

Schema Field	NASA CSV Column	Transformation Rule
material_type	Category	Apply category mapping table below
material_subtype	Composition / Description	Interpret if clearly stated (e.g., thermoplastic, thermoset, reinforced). Otherwise null.
grade	Material_Name (extract)	e.g., "Aluminum 6061-T6" → "T6"; "PEEK 450G" → "450G"
trade_names	Name / Description	Extract if mentioned. Default: []
is_variant	None	false by default. Set true only if clearly a variant of a base material.
base_material_canonical_name	None	null unless is_variant is true

v1.1 — Amendment

v1.3 — Amendment

grade: Extraction Rule for Complex Material Names

Extract grade only when there is an explicit, unambiguous grade or variant designation that stands clearly separate from the base material name — typically a standalone alphanumeric code.

Material_Name example	grade value	Reason
"Aluminum 6061-T6"	"T6"	Unambiguous alphanumeric code
"PEEK 450G"	"450G"	Unambiguous alphanumeric code
"Titanium Grade 5"	"Grade 5"	Explicit grade designation
"Phenolic Impregnated Carbon Ablator"	null	Chemical compound description — no grade token
"Polyimide PMR-15 Composite"	null	Compound name; PMR-15 is the material designation, not a grade

If no discrete grade token is identifiable without interpreting the chemistry, set grade to null.

Category Mapping Table

NASA Category Value	Xtrium material_type Value
Metals	metal
Polymers	polymer
Ceramics	ceramic
Composites	composite
Elastomers	elastomer
Adhesives	adhesive
Glasses	glass
Other	null (or best-effort guess)

v1.1 — Amendment

v1.3 — Amendment

material_type: Decision Rule for Unlisted Category Values

The CSV contains 75+ distinct Category strings not covered by the 8-entry mapping table above. Apply the following decision rule for any unlisted value:

Decision	Action
A reasonable closest-match exists in the 8 known types	Use the closest match (e.g. "Carbon-Phenolics" → composite; "Earth and Stone" → ceramic; "Brick" → ceramic; "Silica-Silicones" → composite)
No reasonable match exists (e.g. "Liquids", "Elements")	Set material_type to null
Category string appears to be a database/source name rather than a material type (e.g. "K&K Associates Materials")	Set material_type to null

In all cases: record the original Category value verbatim in processing_notes. Log every unmapped category in a separate review file and submit alongside the batch for Project Lead review.

5.3 Supplier Context Object

Build the supplier_context object from three NASA columns:

```
"supplier_context": {
  "supplier_canonical_name": "nasa-tpsx",           // from Database_Source
  "supplier_product_id":    null,                   // not in NASA data
  "supplier_grade_name":    "6061-T6",             // extract from Material_Name
  "datasheet_url":          "{URL column value}"
}
```

v1.1 — Amendment

v1.3 — Amendment (extends v1.1)

supplier_canonical_name: Complete Fixed Mapping Table

The dataset contains five distinct Database_Source values. Each maps to a fixed supplier_canonical_name as shown below. Use these as fixed mappings — do not re-derive at runtime:

Database_Source value (in CSV)	supplier_canonical_name (fixed)
TPSX	nasa-tpsx
K&K Associates Materials	kk-associates-materials

NASA JSC Pathfinder Materials	nasa-jsc-pathfinder-materials
NASA Langley Research Center Database	nasa-langley-research-center-database
NASA Ames Thermal Protection Materials	nasa-ames-thermal-protection-materials

If a Database_Source value not in this table is encountered, apply the standard normalization (lowercase, spaces to hyphens, remove & and punctuation) and flag to the Project Lead.

URL Field Rule

The URL column in the NASA TPSX CSV always contains a MaterialDetail URL, e.g.:
`https://tpsx.arc.nasa.gov/MaterialDetail?id=N`
Copy this URL value as-is into ALL URL fields in the output JSON:

Schema Field	URL to Use
supplier_context.datasheet_url	URL column value — always MaterialDetail format
properties[].source.source_url	URL column value — same value
sources[0].source_url	URL column value — same value

No URL construction or format switching is required. Do not modify the URL from the column.

5.4 Processing Notes Assembly

Combine multiple free-text section columns into a single processing_notes string, separated by double newlines:

```
"processing_notes":  
  "Composition: {Composition}\n\nDescription: {Description}\n\nNotes: {Notes}\n\nTRL: {Technical_Readiness_Level}\n\nManufacturer: {Manufacturer}"
```

 **IMPORTANT**

Only include a label prefix (e.g., "Composition:") when the corresponding CSV cell is non-empty. Do not add labels for empty fields. Skip the entire segment if the source cell is blank.

- v1.1 — Amendment
- v1.3 — Amendment
- Parsing Artefact Suppression — Definitive List**

"Blank" includes empty cells AND cells containing only the following parsing artefacts from the PDF-to-CSV export pipeline. Suppress these exactly as you would an empty cell — omit the segment from processing_notes and omit the key from raw:		
Suppress if cell contains (exact or after stripping whitespace)		
"Description:"		
"N/A"		
"n/a"		
"_"		
"\u2014" (em dash)		
"None"		
"Unknown"		
Any cell that is empty or whitespace-only after stripping		
If a cell contains one of these strings alongside other real content (e.g., "N/A — see Description:"), treat the whole cell as real content and include it. The suppression rule applies only to cells where the artefact string is the entire content.		

5.5 Section Column Mapping Summary

CSV Column	Schema Destination	Notes
Composition	chemical_formula (if formula) or processing_notes	Extract chemical formula if recognisable; otherwise append full text to processing_notes
Description	processing_notes	Append with label "Description:"
Notes	processing_notes	Append with label "Notes:"
Technical_Readiness_Level	processing_notes	Append as "TRL: {value}"
Manufacturer	supplier_context.supplier_c anonical_name + processing_notes	Normalise for supplier_context; include raw text in processing_notes
Property_References	standards_referenced + sources[]	Parse standard codes (ASTM, ISO, NASA-TM) into standards_referenced; add as additional source entries

CSV Column	Schema Destination	Notes
General_References	sources[].raw	Include in source citation raw metadata
Point_of_Contact	sources[].raw	Include in source citation raw metadata
Name	material_name (cross-check)	Compare to Material_Name; use as alternate name if different
Database	supplier_context	Usually same as Database_Source; confirm and set accordingly
Last_Modified	sources[].raw	Include in source metadata
Pictures	images[]	Parse URLs into images array (see Section 7)

6. Properties Array Mapping

This is the most critical section. Each material property in the NASA CSV becomes one or more property objects inside the properties array. Follow the steps below precisely.

6.1 Property Object Structure

```
{
  "name": "Density", // Human-readable, spaces
  "canonical_name": "density", // snake_case (see 6.3)
  "value": "2.7", // Always a string
  "numeric_value": 2.7, // Parsed float/int, or null if
non-numeric
  "unit": "g/cm³", // From Units column, or null
  "condition": "at 23°C, STP", // Constructed from STP + instance
info
  "property_category": "physical", // See classification table
  "raw": {
    "Value": "2.7",
    "Units": "g/cm³",
    "Uncertainty": "±0.1",
    "Source": "TPSX Database",
    "STP": "Yes",
    "Reference": "NASA-TM-2015-218675",
    "Last_Modified": "2024-01-15"
  },
  "source": {
    "source_name": "NASA TPSX",
    "source_url": "https://tpsx.arc.nasa.gov/MaterialDetail?id=...",
    "doi": null,
    "tier": "tier1"
  }
}
```


Column Suffix	Maps To	Notes
		Database", empty), source_name = "NASA TPSX".
_STP	raw dict AND condition string	Apply STP mapping table below
_Reference	raw dict only	Also parse into standards_referenced if standard code
_Last_Modified	raw dict only	Source metadata

v1.1 — Amendment**v1.3 — Amendment****value Field: Required String Format**

The SOP states value is "always a string". The CSV exports numeric values as float64, meaning pandas converts the original string representation before the contractor reads it. The required format is:

Situation	value field format	Example
Standard decimal number	Preserve the string as pandas produces it	"2.7" or "288.0"
Large number (float64 output)	Preserve the pandas string output — do not reformat	"83400000.0" (not "8.34e+07")
Non-numeric value (range, text)	Copy as string exactly	"40-60"

Do not attempt to re-format floats back to scientific notation. The numeric_value field carries the actual numeric value for downstream use; the value string is the audit trail of what was read from the CSV. numeric_value should be the parsed float: 83400000.0 (not "8.34e+07").

v1.1 — Amendment**v1.1 — Amendment****Complete STP Column → condition Field Mapping**

The _STP column can contain several distinct values. Apply this mapping table for every STP entry encountered:

STP Column Value	condition Field Output	Notes
"Yes"	"STP"	Standard condition
"TRUE"	"STP"	Treat as equivalent to Yes

"No"	"Non-STP"	Non-standard condition
"nonstp"	"Non-STP"	Normalise to match No output
"assumed/assumed"	"assumed STP"	Assumed standard condition
Absent or blank	null	No condition information

When an instance number is also present, append both: e.g., STP="No" + Instance 1 → condition = "Non-STP, Instance 1". See Section 6.5 for the full instance handling rule.

6.3 Canonical Name Transformation Rules

- Convert entire name to lowercase.
- Replace spaces with underscores.
- Remove possessive apostrophes ('s stays as s).
- Keep existing underscores between words.

NASA Column Base Name	canonical_name Value
Tensile_Strength_Isotropic	tensile_strength_isotropic
Coefficient_of_Thermal_Expansion	coefficient_of_thermal_expansion
Young_s_Modulus	youngs_modulus
Poisson_s_Ratio	poissons_ratio
Thermal_Conductivity_Thru_the_Thickness	thermal_conductivity_thru_the_thickness

6.4 Property Category Classification

property_category	NASA Property Names (examples)
mechanical	Tensile_Strength, Tensile_Modulus, Compressive_Strength, Shear_Modulus, Poisson_s_Ratio, Bending_Strength, Bearing_Strength, Flexural_Modulus, Fracture_Toughness, Fatigue_Strength, Impact_Strength, Hardness
thermal	Thermal_Conductivity, Specific_Heat, Coefficient_of_Thermal_Expansion, Melting_Temperature, Boiling_Temperature, Glass_Transition_Temperature, Emissivity, Thermal_Diffusivity
electrical	Electrical_Resistivity, Electrical_Conductivity, Dielectric_Constant

property_category	NASA Property Names (examples)
physical	Density, Molecular_Weight, Viscosity
optical	Refractive_Index, Transmittance, Reflectance, Absorptance, Solar_Absorptance, Infrared_Emittance
chemical	Oxygen_Recombination_Coefficient, Nitrogen_Recombination_Coefficient, Air_Recombination_Coefficient, Outgassing_Rate, Total_Mass_Loss, Collected_Volatile_Condensable_Material
environmental	Effective_Heat_of_Ablation, Char_Yield, Pyrolysis_Gas_Enthalpy
processing	Multiple_Use_Temperature_Limit, Single_Use_Temperature_Limit

6.5 Handling Multiple Property Instances

When the same property appears with numeric suffixes, create one property object per instance. The instance number goes into the condition field and into raw.

v1.1 — Amendment

v1.3 — Amendment

Instance Labelling: Count-Based, Not Suffix-Based

Numeric suffixes in CSV column names are global block counters across the entire row — they do not reliably indicate how many instances of a specific property exist. Instance labels must be assigned based on the actual count of occurrences of the same base property name within that material’s row, not on the column suffix number.

Situation	condition label	Reason
Property base name appears once in the row (e.g. only Tensile_Yield_Strength_Isotropic_2_Value exists, no _1_ or _3_)	null (no instance label)	Single occurrence — no ambiguity to resolve
Property base name appears two or more times (e.g. Thermal_Conductivity_1_Value AND Thermal_Conductivity_2_Value)	"Instance 1", "Instance 2", ... in ascending suffix order	Multiple readings of same property — label sequentially

Example: Tensile_Strength_Isotropic_1_Value, Tensile_Yield_Strength_Isotropic_2_Value, and Tensile_Modulus_In_Plane_3_Value are three different properties each appearing once. None receives an instance label — condition is null for all three (unless STP adds a condition).

EXAMPLE — Two instances of Thermal Conductivity (same base name, two occurrences)

CSV columns present: Thermal_Conductivity_1_Value, Thermal_Conductivity_1_Units, Thermal_Conductivity_1_Source, Thermal_Conductivity_2_Value, Thermal_Conductivity_2_Units

Same base name appears twice – create two property objects:

First object: condition = "Instance 1", raw.instance = "1"

Second object: condition = "Instance 2", raw.instance = "2"

7. Images Mapping

The Pictures CSV column may contain image URLs. Each URL becomes one entry in the images array.

```
"images": [  
  {  
    "image_url": "https://tpsx.arc.nasa.gov/images/material_123.jpg",  
    "image_data": null,  
    "image_type": null  
  }  
]
```

images Field	CSV Source	Rule
image_url	Pictures column	Extract URL. If column contains multiple URLs, create one object per URL.
image_data	N/A	Always null — binary data is not in the CSV export
image_type	N/A	null unless type is determinable (e.g., "microstructure")

 IMPORTANT

If the Pictures column is empty or contains non-URL data, set images to an empty array [].

8. Source Citation Mapping

Every material record must include at least one entry in the sources array. Use the following tiers:

Tier	Definition	Examples
tier1	Government / official database sources	NASA TPSX, NASA technical memoranda

Tier	Definition	Examples
tier2	Supplier datasheets	Manufacturer-published material data
tier3	General web / unverified	All other sources

8.1 Base Source Entry (Always Required)

```
"sources": [  
  {  
    "source_name": "NASA TPSX Materials Database",  
    "source_url": "{URL column value}",  
    "doi": null,  
    "tier": "tier1",  
    "raw": {  
      "General_References": "{General_References column value or omit if empty}",  
      "Point_of_Contact": "{Point_of_Contact column value or omit if empty}",  
      "Last_Modified": "{Last_Modified column value or omit if empty}"  
    }  
  }  
]
```

v1.1 — Amendment

v1.3 — Amendment

sources[0].raw: Structure and Population Rule

Section 5.5 maps General_References, Point_of_Contact, and Last_Modified to sources[].raw. The raw field is a structured object with separate keys — not a concatenated string.

CSV Column	Key in sources[0].raw	Rule
General_References	"General_References"	Include if non-empty and not a suppressed artefact
Point_of_Contact	"Point_of_Contact"	Include if non-empty and not a suppressed artefact
Last_Modified	"Last_Modified"	Include if non-empty and not a suppressed artefact

Apply the same key-omission rule as property-level raw: if a CSV cell is empty or contains a suppressed artefact (see §5.4), omit that key entirely from sources[0].raw. If all three cells are empty, omit the raw field from sources[0] altogether.

v1.1 — Amendment

v1.1 — Amendment

Top-Level source_name Must Use the Long Form

The sources[0].source_name at the top level must always be "NASA TPSX Materials Database" (the full publication name). This is intentionally different from the property-level source.source_name which uses the short form "NASA TPSX".

Summary of the two-level rule:

Level	Correct source_name
Top-level sources[0]	"NASA TPSX Materials Database"
properties[].source	"NASA TPSX"

8.2 Parsing Property_References into Additional Sources

If the Property_References column contains citation strings, parse them as follows:

- Standard codes (ASTM, ISO) → add to standards_referenced array AND create a source entry with tier1.
- NASA technical memo codes (NASA-TM-xxxx) → create a source entry with tier1.
- All reference text → also copy into sources[0].raw for full fidelity.

EXAMPLE — Property_References parsing

Input: "NASA-TM-2015-218675, ASTM D638, ISO 527-1"

standards_referenced: ["ASTM D638", "ISO 527-1"]

sources: adds entry for NASA-TM-2015-218675 with tier = "tier1"

9. Step-by-Step Transformation Workflow

Follow these steps in sequence for every CSV row (one material record).

Step	Name	Action
Step 1	Load Row	Read a single CSV row representing one material record.
Step 2	Extract Base Fields	Pull Id, Material_Name, Database_Source, Category, URL.
Step 3	Map Top-Level Fields	Populate material_name, canonical_name, material_type, grade, supplier_context using Section 5 rules.
Step 4	Identify Property Columns	Scan remaining columns and group by base property name (e.g., all Density_* columns together).
Step 5	Build Property Objects	For each property group, identify base instance + any numbered instances. Create one property object per instance using Section 6 rules.

Step	Name	Action
Step 6	Classify & Populate	Set property_category, canonical_name, condition, raw, and source for each property object. Append to properties array.
Step 7	Map Section Columns	Process Composition, Description, Notes, Manufacturer, Technical_Readiness_Level into processing_notes per Section 5.4.
Step 8	Parse References	Extract standards into standards_referenced; build additional source entries from Property_References and General_References per Section 8.
Step 9	Handle Images	Parse Pictures column into images array per Section 7.
Step 10	Set Null Defaults	Set all remaining schema fields that have no data to null or [] as appropriate.
Step 11	Validate JSON	Run the validation checklist in Section 13 before saving the file.
Step 12	Save Output	Write final JSON to output file named {canonical_name}.json.

10. Handling Missing & Edge-Case Data

Situation	Required Action
Empty CSV cell (non-critical field)	Set corresponding schema field to null. Do not include the key in raw if empty.
Value is a range (e.g., "40-60")	Keep as string in value. Set numeric_value to null. Preserve range string in raw.
New property not listed in this document	Follow the same pattern: create property object, classify best-guess category, include all attributes in raw.
Property with no Units column	Set unit to null. Add a note in raw that units were not provided.
Unrecognised NASA Category	Set material_type to null. Record original Category value in processing_notes.
Multiple URLs in Pictures column	Create one images entry per URL.
Empty Pictures column	Set images to [].

Situation	Required Action
Composition is a formula	Place in chemical_formula and also include in processing_notes.
Do not clean/normalise property values	Copy values exactly as they appear in CSV. Normalisation happens downstream.

v1.1 — Amendment

v1.1 — Amendment

raw Dict: Key Omission vs. Null — Explicit Example

The rule "Do not include the key in raw if empty" means the key must be completely absent — not present with a null value. The distinction matters for downstream pipeline validation.

CORRECT — key omitted entirely (Source column was empty):

```
"raw": {
  "Value": "2.7",
  "Units": "g/cm³"
}
```

INCORRECT — key present with null value:

```
"raw": {
  "Value": "2.7",
  "Units": "g/cm³",
  "Source": null      ← DO NOT do this
}
```

This rule applies to every key in every raw dict. If the source CSV cell is blank, the corresponding key must not appear in raw at all.

11. Complete Worked Examples

11.1 Simple Metal Material — Aluminum 6061-T6

Input CSV

```
Id: 1
Material_Name: Aluminum 6061-T6
Database_Source: TPSX
Category: Metals
URL: https://tpsx.arc.nasa.gov/MaterialDetail?id=1
Composition: Al-Mg-Si alloy
Description: Aerospace-grade aluminum alloy
Density_Value: 2.7
Density_Units: g/cm³
Density_Source: TPSX Database
Tensile_Strength_Isotropic_1_Value: 310
Tensile_Strength_Isotropic_1_Units: MPa
Tensile_Strength_Isotropic_1_Source: ASTM B209
```

Output JSON

```
{
  "material_id": null,
  "material_name": "Aluminum 6061-T6",
  "canonical_name": "aluminum-6061-t6",
  "cas_number": null, "uns_number": null, "icsd_id": null,
  "inchikey": null, "external_id_type": null,
  "chemical_formula": null,
  "material_type": "metal",
  "material_subtype": null,
  "grade": "T6",
  "trade_names": [], "is_variant": false,
  "base_material_canonical_name": null,
  "supplier_context": {
    "supplier_canonical_name": "nasa-tpsx",
    "supplier_product_id": null,
    "supplier_grade_name": "6061-T6",
    "datasheet_url": "https://tpsx.arc.nasa.gov/MaterialDetail?id=1"
  },
  "properties": [
    {
      "name": "Density", "canonical_name": "density",
      "value": "2.7", "numeric_value": 2.7,
      "unit": "g/cm³", "condition": null,
      "property_category": "physical",
      "raw": { "Value": "2.7", "Units": "g/cm³", "Source": "TPSX Database" },
      "source": { "source_name": "NASA TPSX",
        "source_url": "https://tpsx.arc.nasa.gov/MaterialDetail?id=1",
        "doi": null, "tier": "tier1" }
    },
    {
      "name": "Tensile Strength Isotropic",
      "canonical_name": "tensile_strength_isotropic",
      "value": "310", "numeric_value": 310,
      "unit": "MPa", "condition": "Instance 1",
      "property_category": "mechanical",
      "raw": { "Value": "310", "Units": "MPa", "Source": "ASTM B209", "instance":
"1" },
      "source": { "source_name": "NASA TPSX",
        "source_url": "https://tpsx.arc.nasa.gov/MaterialDetail?id=1",
        "doi": null, "tier": "tier1" }
    }
  ],
  "standards_referenced": ["ASTM B209"],
  "compliance_references": [], "applications_mentioned": [],
  "processing_notes": "Composition: Al-Mg-Si alloy\n\nDescription: Aerospace-grade
aluminum alloy",
  "hazard_flags": [], "availability_notes": null,
  "images": [],
  "sources": [{ "source_name": "NASA TPSX Materials Database",
    "source_url": "https://tpsx.arc.nasa.gov/MaterialDetail?id=1",
    "doi": null, "tier": "tier1" }]
}
```

11.2 Multiple Instances — Carbon Fiber Composite

Input CSV (relevant columns)

```
Material_Name: Carbon Fiber Composite
Thermal_Conductivity_Thru_the_Thickness_Value: 0.5
Thermal_Conductivity_Thru_the_Thickness_Units: W/m·K
```

```
Thermal_Conductivity_Thru_the_Thickness_STP: Yes
Thermal_Conductivity_Thru_the_Thickness_1_Value: 0.6
Thermal_Conductivity_Thru_the_Thickness_1_Units: W/m·K
Thermal_Conductivity_Thru_the_Thickness_1_STP: No
Thermal_Conductivity_In_Plane_Value: 15.0
Thermal_Conductivity_In_Plane_Units: W/m·K
```

Resulting properties Array (3 objects)

```
{ "name": "Thermal Conductivity Thru the Thickness",
  "canonical_name": "thermal_conductivity_thru_the_thickness",
  "value": "0.5", "numeric_value": 0.5, "unit": "W/m·K",
  "condition": "STP", "property_category": "thermal",
  "raw": { "Value": "0.5", "Units": "W/m·K", "STP": "Yes" }, ... },

{ "name": "Thermal Conductivity Thru the Thickness",
  "canonical_name": "thermal_conductivity_thru_the_thickness",
  "value": "0.6", "numeric_value": 0.6, "unit": "W/m·K",
  "condition": "Non-STP, Instance 1", "property_category": "thermal",
  "raw": { "Value": "0.6", "Units": "W/m·K", "STP": "No", "instance": "1" }, ... },

{ "name": "Thermal Conductivity In Plane",
  "canonical_name": "thermal_conductivity_in_plane",
  "value": "15.0", "numeric_value": 15.0, "unit": "W/m·K",
  "condition": null, "property_category": "thermal",
  "raw": { "Value": "15.0", "Units": "W/m·K" }, ... }
```

12. Property Groups Reference

The following 133 property groups are known to exist in the NASA TPSX dataset. If you encounter a group not on this list, follow the same property object pattern and flag it for review by the Project Lead.

Mechanical Properties (Groups 1–29)

#	Property Group Name
1	Tensile_Strength_Isotropic
2	Tensile_Strength_Thru_the_Thickness
3	Tensile_Strength_In_Plane
4	Tensile_Yield_Strength_Isotropic
5	Tensile_Modulus_Isotropic
6	Tensile_Modulus_In_Plane
7	Tensile_Modulus_Thru_the_Thickness
8	Tensile_Strain_to_Failure_Isotropic

#	Property Group Name
9	Tensile_Strain_to_Failure_In_Plane
10	Tensile_Strain_to_Failure_Thru_the_Thickness
11	Compressive_Strength_Isotropic
12	Compressive_Strength_In_Plane
13	Compressive_Strength_Thru_the_Thickness
14	Compressive_Modulus_Isotropic
15	Compressive_Modulus_In_Plane
16	Compressive_Strain_to_Failure_Isotropic
17	Compressive_Strain_to_Failure_In_Plane
18	Compressive_Strain_to_Failure_Thru_the_Thickness
19	Shear_Modulus_Isotropic
20	Shear_Modulus_In_Plane
21	Shear_Strength_Isotropic
22	Shear_Strength_In_Plane
23	Poisson_s_Ratio_Isotropic
24	Poisson_s_Ratio_In_Plane
25	Bending_Strength_Isotropic
26	Bending_Strength_In_Plane
27	Bending_Strength_Thru_the_Thickness
28	Flexural_Modulus_Isotropic
29	Bearing_Strength_Isotropic

Thermal Properties (Groups 30–44)

#	Property Group Name
30	Thermal_Conductivity_Isotropic

#	Property Group Name
31	Thermal_Conductivity_In_Plane
32	Thermal_Conductivity_Thru_the_Thickness
33	Specific_Heat
34	Coefficient_of_Thermal_Expansion_Isotropic
35	Coefficient_of_Thermal_Expansion_In_Plane
36	Coefficient_of_Thermal_Expansion_Thru_the_Thickness
37	Melting_Temperature
38	Boiling_Temperature
39	Glass_Transition_Temperature
40	Emissivity
41	Effective_Heat_of_Ablation
42	Multiple_Use_Temperature_Limit
43	Single_Use_Temperature_Limit
44	Thermal_Diffusivity_Isotropic

Electrical (45–47), Physical (48–50), Optical (51–56)

#	Property Group Name
45	Electrical_Resistivity
46	Electrical_Conductivity_Isotropic
47	Dielectric_Constant_Isotropic
48	Density
49	Molecular_Weight
50	Viscosity
51	Refractive_Index
52	Transmittance

#	Property Group Name
53	Reflectance
54	Absorptance
55	Solar_Absorptance
56	Infrared_Emittance

Chemical (57–62), Environmental/Ablation (63–65), Other (66–69)

#	Property Group Name
57	Oxygen_Recombination_Coefficient (variants 1–9)
58	Nitrogen_Recombination_Coefficient (variants 1–9)
59	Air_Recombination_Coefficient (variants 1–9)
60	Outgassing_Rate
61	Total_Mass_Loss
62	Collected_Volatile_Condensable_Material
63	Effective_Heat_of_Ablation (Environmental)
64	Char_Yield
65	Pyrolysis_Gas_Enthalpy
66	Hardness_Isotropic
67	Fracture_Toughness_Isotropic
68	Impact_Strength_Isotropic
69	Fatigue_Strength_Isotropic

13. Pre-Submission Validation Checklist

Run this checklist against every JSON file before submitting a batch. Do not submit any file that fails one or more items. Flag failures to your QA Reviewer.

How to use this checklist

For each completed JSON file, go through every item below. If all items pass, the file is ready for submission. If any item fails, correct the issue and re-validate before submitting.

Required Fields

- material_name is present and non-empty.
- properties array contains at least one entry.
- sources array contains at least one entry.
- All property objects have name, value, raw, and source populated.

Data Integrity

- All 1,287 CSV columns are accounted for — none discarded.
- raw field in each property object contains all CSV attributes for that property.
- No data invented or inferred — only null where source data is absent.
- numeric_value is a parsed float/int, or null if value is non-numeric.
- No key in any raw dict is set to null — absent data means the key is omitted entirely.

Formatting

- null used for all missing singular fields (not empty string "").
- Empty arrays [] used for all missing list/array fields.
- canonical_name is lowercase-hyphenated (no uppercase, no spaces).
- All property canonical_names are snake_case (no hyphens).

Property Mapping

- Property categories assigned correctly per the table in Section 6.4.
- Multiple property instances are separate objects with correct condition values.
- STP condition values use the mapping table in Section 6.2 — not the raw STP string.
- Source tier set to "tier1" for all NASA TPSX data.
- standards_referenced populated from Property_References where applicable.

Source Names

- properties[].source.source_name = the _Source column value if it is a named external standard (ASTM, ISO, NASA-TM). Otherwise "NASA TPSX".
- sources[0].source_name = "NASA TPSX Materials Database" (long form, always).
- Generic _Source values such as "measured" or "TPSX Database" never appear as source_name — use "NASA TPSX" instead.
- No mixing of the long form into property-level source objects.

Output File

- File is valid, parsable JSON.
- File named {canonical_name}.json.

- One file per material record.

14. Frequently Asked Questions

Question	Answer
What if a property has no Units column?	Set unit to null. Add a note in raw that units were not provided: "Units": "not provided".
What if Value is a range like "40-60"?	Keep the range as a string in value. Set numeric_value to null. Preserve the range string verbatim in raw.
What if we encounter a new property not in Section 12?	Follow the same property object pattern. Classify the best-fit category. Include all attributes in raw. Flag the new property to the Project Lead for catalogue update.
Should we clean or normalise property values?	No. Copy values exactly as they appear in the CSV. Normalisation happens in later pipeline stages.
What about empty CSV cells?	Do not include the key in the raw dict. If it is a critical field (like Value), set the corresponding schema field to null.
The _Source column says "measured" — what do I put in source.source_name?	"measured" is a measurement method, not a named data source. It goes into raw.Source only. Set source.source_name to "NASA TPSX". Only named external standards (ASTM, ISO, NASA-TM codes) flow through from _Source into source.source_name directly.
Which _Source values count as a named external standard?	Standard codes that follow a recognised format: ASTM XXXX, ISO XXX-X, NASA-TM-XXXX, NASA-CR-XXXX. Values like "measured", "TPSX", "TPSX Database", "lab measurement", or any blank/generic description are not named standards — use "NASA TPSX" as the fallback.
Why does source_name differ between property level and sources[]?	The two levels intentionally use different strings: "NASA TPSX" at property level and "NASA TPSX Materials Database" at the top-level sources array. See §6.1 and §8.1.
What does STP = "assumed/assumed" mean for the condition field?	Map it to "assumed STP". See the complete STP mapping table in §6.2.

Question	Answer
A CSV cell contains only "Description:" or "N/A" — is it blank?	Yes. These are parsing artefacts and must be treated as blank. See the suppression list in §5.4. Omit the segment from processing_notes and omit the key from raw.
A property column has suffix _2_ but is the only instance of that property in this row. Does it get "Instance 2" in condition?	No. Instance labels are based on the actual count of occurrences of the same base property name in the row. If the property appears only once, condition is null regardless of the suffix number. See §6.5.
The value field comes out as 83400000.0 from pandas — should I reformat it?	No. Preserve the string as pandas produces it. Do not reformat to scientific notation. Set numeric_value to the parsed float (83400000.0). See §6.2.
A category string is not in the mapping table. What material_type do I use?	Apply the closest match from the 8 known types where possible. If no reasonable match exists, set null. Always record the original Category in processing_notes and log unmapped categories in the batch review file. See §5.2.
The Composition cell contains a formula mixed with prose. Does it go in chemical_formula?	No. Only extract to chemical_formula when the entire cell content is a formula string with no prose. Mixed cells go to processing_notes in full. See §5.1.
Should sources[0] include a raw field?	Yes. sources[0].raw is a structured object with General_References, Point_of_Contact, and Last_Modified as separate keys. Omit any key whose CSV cell is empty or a suppressed artefact. If all three are empty, omit raw entirely. See §8.1.
Who do we contact for unresolved mapping questions?	Escalate to the Project Lead. Document the question, the source column, and the attempted mapping before escalating.

15. Document Control

Version	Date	Author	Changes
1.0	March 29, 2026	Xtrium Data Team	Initial release
1.1	June 3, 2026	Xtrium Data Team	Q2: URL field rule added to §5.3. Q3: supplier_canonical_name derivation added to §5.3. Q4: source_name two-level rule in §6.1 and §8.1. Q5: Complete STP mapping table in §6.2.

Version	Date	Author	Changes
			Q6: raw dict key-omission example in §10.
1.2	June 3, 2026	Xtrium Data Team	Q4 refinement: two-tier source_name fallback logic documented in §6.1 and §6.2. Named standards flow through from _Source; generic values fall back to "NASA TPSX". §13 and §14 updated.
1.3	June 3, 2026	Xtrium Data Team	Phase 1 Inquiry (8 items): Q1 — sources[0].raw structured object added to §8.1. Q2 — Complete 5-value supplier_canonical_name mapping table in §5.3. Q3 — Parsing artefact suppression list added to §5.4. Q4 — Count-based instance labelling rule in §6.5. Q5 — value field string format rule in §6.2. Q6 — material_type decision rule for unlisted categories in §5.2. Q7 — chemical_formula extraction threshold in §5.1. Q8 — grade extraction rule for complex names in §5.2. §13 checklist and §14 FAQ updated.

For questions or corrections regarding this document, contact the Xtrium Data Team. All suggested revisions must be approved by the Project Lead before taking effect.

— END OF DOCUMENT —